05

Part 5 · Storage Layer & File Layout

File formats, small files and Delta maintenance: the cheapest speed-ups.

- File formats
- Schema handling
- Compression
- Small files
- Compaction
- Delta OPTIMIZE
- Z-ordering
- VACUUM

5.1 CSV vs JSON vs Parquet vs Delta Lake

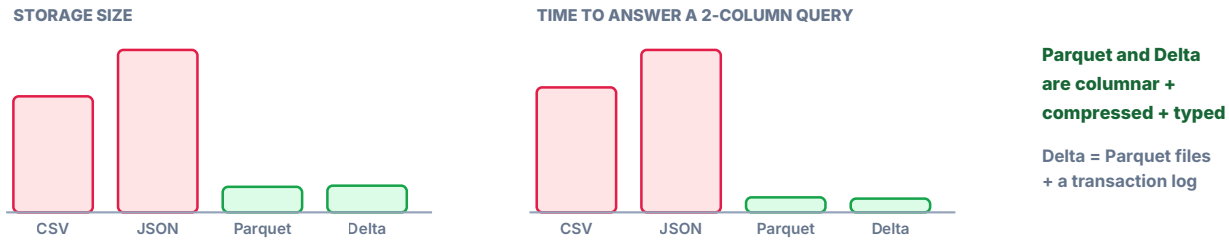
★★★

IN ONE LINE

Columnar, compressed formats with schemas (Parquet, Delta) beat row text formats for analytics.

File format is one of the cheapest big wins: switching from CSV/JSON to a **columnar, compressed, schema-carrying** format can cut storage by 70–90% and make queries many times faster, because column pruning, predicate pushdown and compression all become possible.

SAME 100 GB DATASET · illustrative relative sizes and scan times



Feature	CSV	JSON	Parquet	Delta Lake
Layout	row text	row text	columnar binary	Parquet + log
Schema stored	no	no (inferred)	yes	yes (enforced)
Column pruning / pushdown	no	no	yes	yes + data skipping
Compression	poor	poor	excellent	excellent
ACID / time travel / MERGE	no	no	no	YES

Figure 5.1 — Formats compared

Format	Use it for	Avoid for
CSV	Raw ingestion from external systems, tiny files, spreadsheets	Analytics: no schema, no types, no pruning, slow parsing
JSON	Semi-structured landing data, APIs, nested events	Large analytics: verbose, schema inference, no pruning
Parquet	Analytical storage: columnar, compressed, splittable, widely supported	Row-level updates / deletes, ACID needs
Delta Lake	Lakehouse tables: ACID, MERGE, time travel, schema enforcement, OPTIMIZE / data skipping	Systems that cannot read Delta (use UniForm / export if needed)

INTERVIEW ANGLE

The standard pattern: **land raw CSV/JSON in Bronze** (as received), then **convert to Delta/Parquet in Silver** with an explicit schema. Never run repeated analytics on raw CSV.

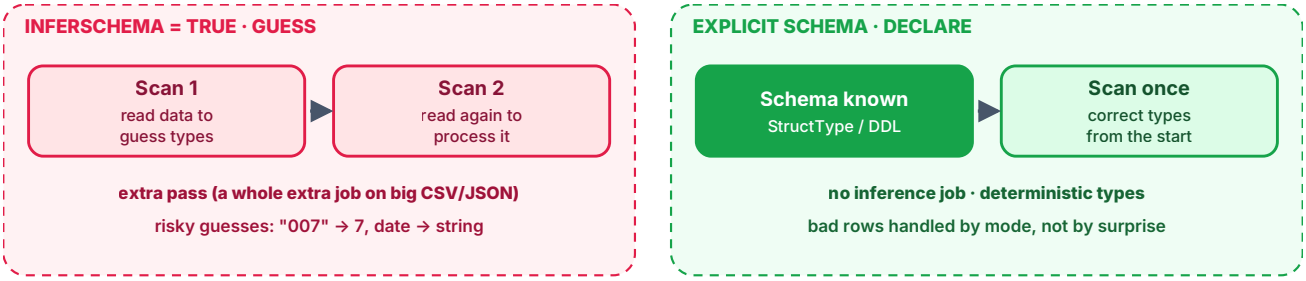
5.2 Explicit Schema Definition

★★★

IN ONE LINE

Declare column types up front instead of making Spark infer them with an extra scan.

When Spark reads CSV or JSON it can **infer** the schema by scanning the data. Explicit schemas are faster, safer and give full control of data types.




```
schema = StructType([
StructField('order_id', LongType(), False),
StructField('amount', DecimalType(12,2)),
StructField('order_ts', TimestampType())])
df = spark.read.schema(schema).option('mode','FAILFAST').csv(path)
# or DDL string: .schema('order_id BIGINT, amount DECIMAL(12,2), order_ts TIMESTAMP')
```

Figure 5.2 — Schema inference costs a scan and can guess wrong

Problem with inference	Explicit schema fixes it
Extra pass over the data (slow on big files)	No inference job
Wrong guesses: zip codes as int, dates as string, long ids as int	You choose `StringType`, `DateType`, `LongType`, `DecimalType`
Schema can change between runs if samples differ	Stable, repeatable types
Corrupt rows silently become nulls	Pick a read `mode`: **PERMISSIVE** (default), **DROPMALFORMED**, **FAILFAST**

INTERVIEW ANGLE

For **Parquet / Delta / ORC** the schema is stored in the file, so no inference scan is needed. The advice matters mostly for **CSV and JSON**. For streaming file sources a schema is **required** (or use Auto Loader schema inference, 8.4).

5.3 Compression: Snappy, Zstandard and Others

★★☆

IN ONE LINE

Compression trades a little CPU for much less I/O; Snappy and zstd are the usual choices.

Compression shrinks files so less data is read from disk and moved across the network, at the price of some CPU to compress and decompress. For analytics the trade is almost always worth it.

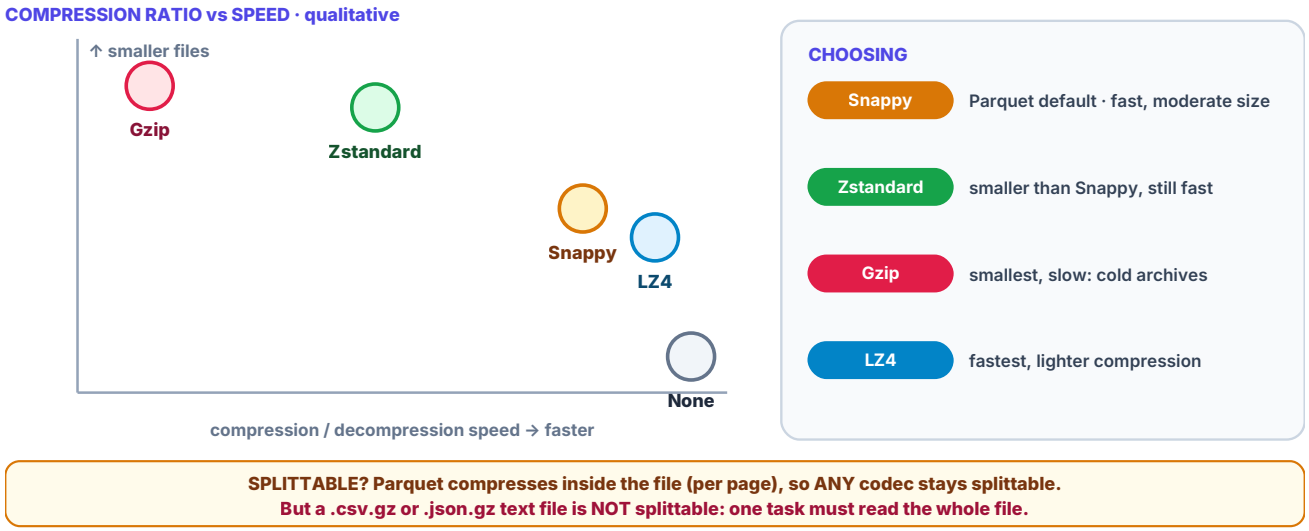


Figure 5.3 — Codec trade-offs

Codec	Ratio	Speed	Typical use
Snappy	Moderate	Very fast	Default for Parquet in Spark; good all-rounder
Zstandard (zstd)	Better than Snappy	Fast	Great modern choice when storage / IO cost matters
Gzip	High	Slow	Archives; avoid for hot analytical tables
LZ4	Lower	Fastest	Shuffle / short-lived data

SETTING THE CODEC

```
# write with a chosen codec
df.write.option('compression', 'zstd').parquet('/lake/silver/orders')

# or session-wide
spark.conf.set('spark.sql.parquet.compression.codec', 'zstd')
```

COMMON TRAP

Big **gzip-compressed CSV/JSON** files cannot be split, so one task reads each file serially — a classic hidden bottleneck. Convert such inputs to Parquet/Delta early, or split them into many files.

5.4 The Small Files Problem

★★★

IN ONE LINE

Too many tiny files add per-file overhead that slows analytical reads and writes.

The **small files problem** is having very many tiny files (KBs to a few MBs) instead of fewer large ones. Each file adds fixed overhead — listing, opening, reading metadata, scheduling a task — that dwarfs the useful work. It is one of the most common causes of slow lakehouse queries.

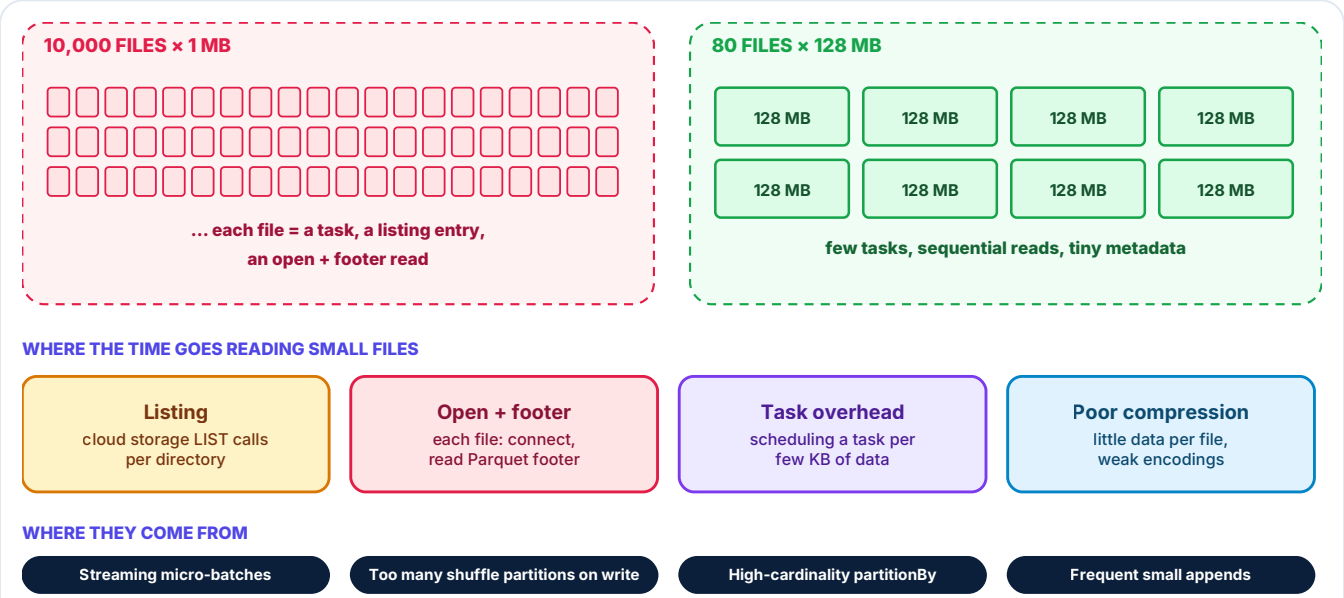


Figure 5.4 — Many tiny files vs a few right-sized files

Cause	Prevention
Writing with too many shuffle partitions (each writes its own file)	`coalesce` / `repartition` before write; AQE coalescing
`partitionBy` on a high-cardinality column	Better partition column (2.9); cluster instead
Streaming micro-batches every few seconds	Larger trigger intervals; Delta auto-compaction / optimized writes
Frequent tiny appends / upserts	Batch writes; periodic `OPTIMIZE`

REMEMBER

Target file size: roughly **128 MB – 1 GB** per file for analytical tables. Symptoms in the UI: thousands of tiny tasks, long job-start listing time, many sub-MB files in the table.

5.5 File Compaction

★★☆

IN ONE LINE

Merging many small files into fewer, larger ones to speed up reads.

File compaction (also *bin-packing*) merges many small files into fewer large ones. For plain Parquet you do it yourself by rewriting the data; Delta Lake offers built-in commands.

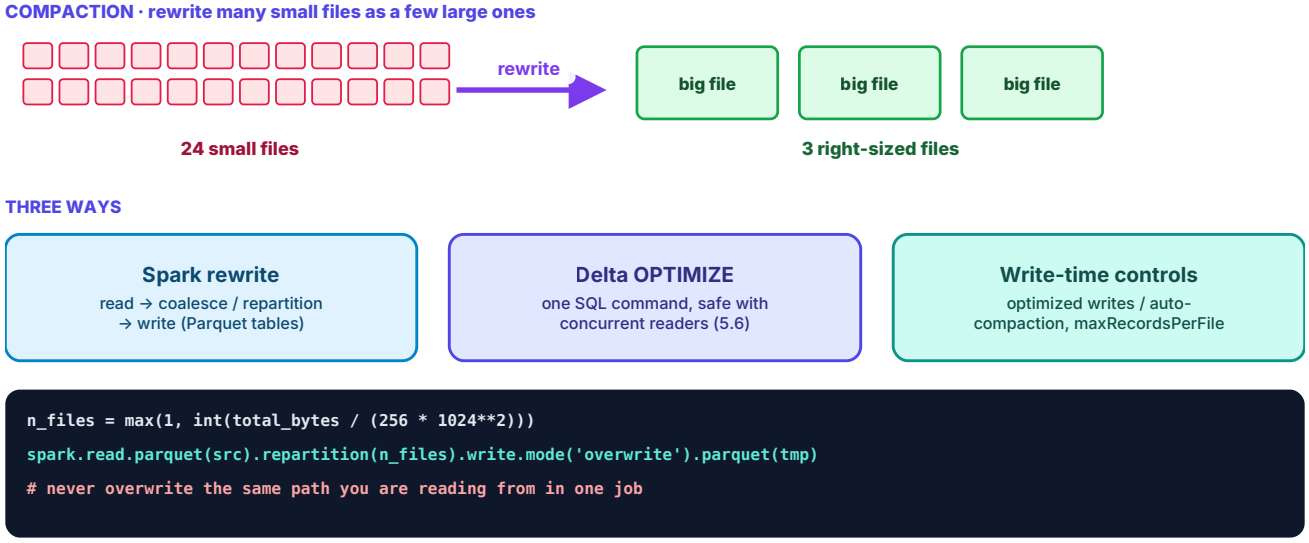


Figure 5.5 — Rewrite small files into large ones

COMPACTION AT WRITE TIME

```
# control file count / size when writing
(df.repartition(16)                       # ~16 output files
   .write.option('maxRecordsPerFile', 5_000_000)   # cap rows per file
   .mode('append').parquet('/lake/silver/events'))

# Delta: let Spark pack files while writing
spark.conf.set('spark.databricks.delta.optimizeWrite.enabled', True)
spark.conf.set('spark.databricks.delta.autoCompact.enabled', True)
```

Approach	Pros	Cons
Rewrite with Spark	Works for any format	Manual; careful with overwrite-in-place; readers can see partial state on plain Parquet
Delta `OPTIMIZE`	Transactional, incremental, one command	Delta only; uses compute
Optimised write / auto compaction	Prevents small files automatically	Slightly slower writes; Databricks features

INTERVIEW ANGLE

Schedule compaction **off-peak** for hot tables, and compact only partitions that changed (e.g. `WHERE date = current_date() - 1`).

5.6 Delta Lake OPTIMIZE

★★★

IN ONE LINE

Compacts small Delta files into larger ones, transactionally and incrementally.

OPTIMIZE is the Delta Lake command that compacts a table's small files into larger ones (target around **1 GB**; configurable). It rewrites files in a transaction, so queries keep working during the operation.

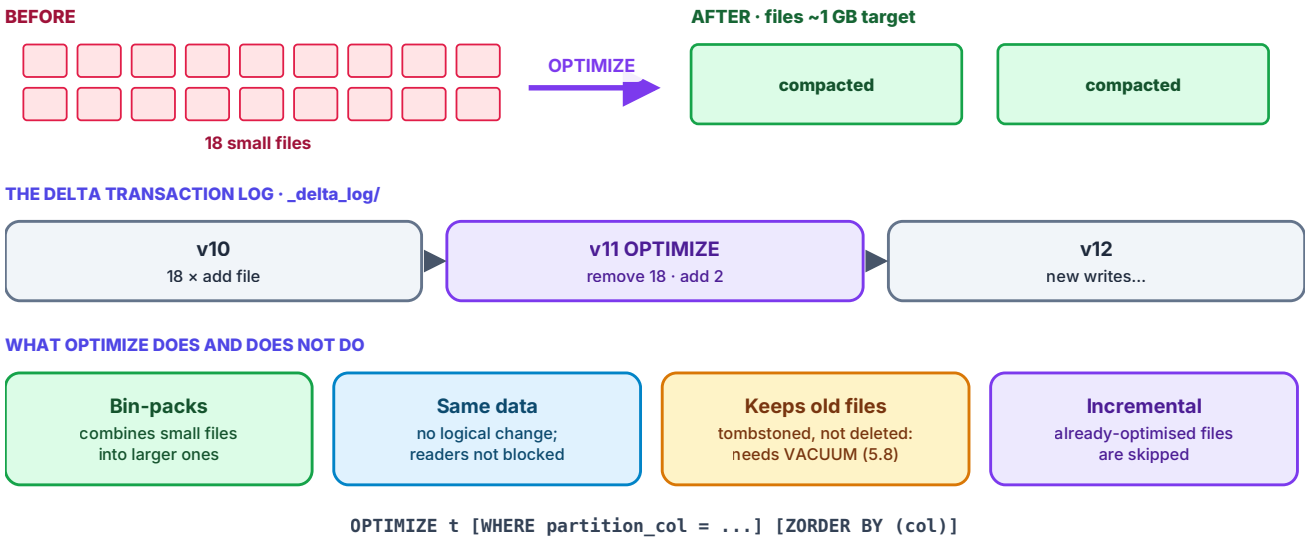


Figure 5.6 — OPTIMIZE bin-packs files and records it in the transaction log

USING OPTIMIZE

```
-- compact the whole table
OPTIMIZE sales.orders;

-- compact only recent partitions (cheaper)
OPTIMIZE sales.orders WHERE order_date >= '2026-10-01';

-- compact and co-locate by a filter column (5.7)
OPTIMIZE sales.orders ZORDER BY (customer_id);
```

Fact	Detail
Idempotent / incremental	Re-running skips files that are already large enough.
Does not delete old files	Previous small files remain for time travel until **VACUUM**.
Costs compute	It reads and rewrites data: run on a schedule, filter by partition.
Automation	**Auto compaction** and **optimized writes** reduce the need; **predictive optimization** on Unity Catalog managed tables can run OPTIMIZE for you (7.7).

INTERVIEW ANGLE

Likely question: *"Our Delta table queries got slow over time."* Answer: small files from frequent appends / streaming → run `OPTIMIZE` (maybe with `ZORDER`), then `VACUUM`, and enable auto-compaction going forward.

5.7 Z-Ordering and Data Skipping

★★★

IN ONE LINE

File-level min/max statistics let Delta skip files; Z-ordering makes those ranges narrow.

Delta Lake stores **min/max statistics for each column of each file** in its transaction log. A query with a filter first checks the log and **skips files** whose ranges cannot match: this is **data skipping**. It only works well if related values are stored together, and that is what **Z-ordering** does: it sorts data along several columns at once so each file covers a narrow range of each.

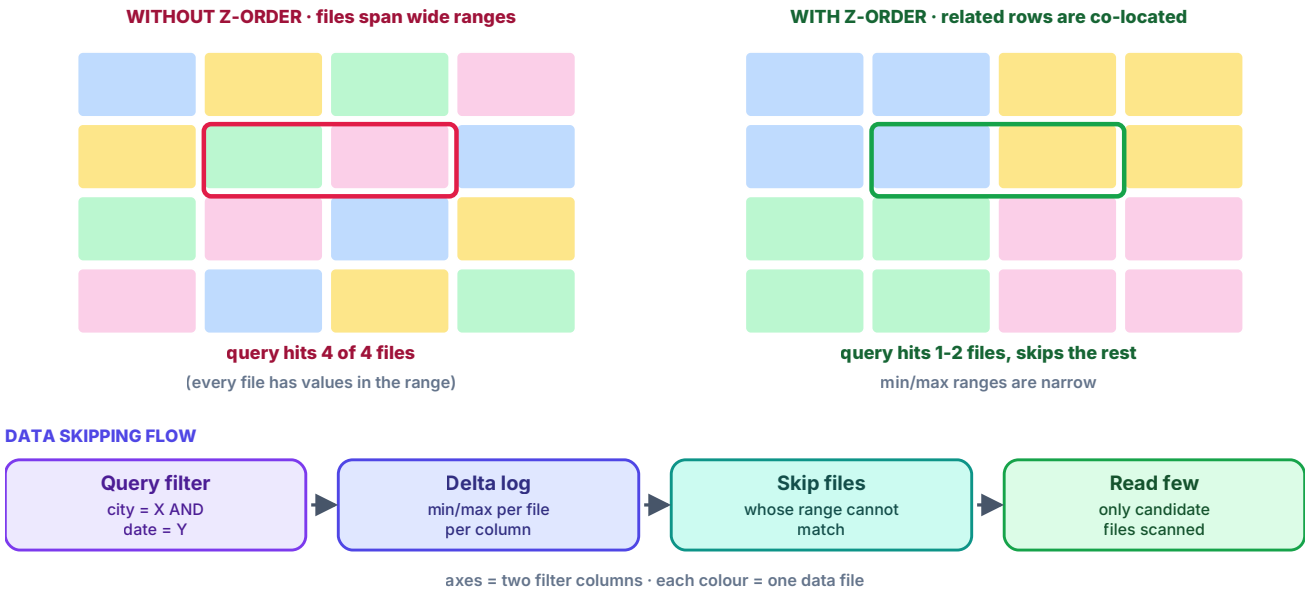


Figure 5.7 — Z-order groups similar values into the same files so more files can be skipped

Z-ORDER A DELTA TABLE

```
OPTIMIZE sales.orders ZORDER BY (customer_id, order_date);

SELECT * FROM sales.orders WHERE customer_id = 42;   -- skips most files
```

Guideline	Reason
Pick high-cardinality columns that queries filter on (customer_id, device_id)	Low-cardinality columns are better as partitions
Use 1–3 columns	Effectiveness drops as you add more
Stats are kept for the first 32 columns by default	Put filter columns early or tune `delta.dataSkippingNumIndexedCols`
Z-order is applied when you run OPTIMIZE	New data arrives unordered until the next OPTIMIZE
Do not Z-order on partition columns	Partition pruning already covers them

INTERVIEW ANGLE

On new Databricks tables, **liquid clustering** (7.7) is the recommended, more flexible replacement for partitioning + Z-order. Know both: Z-order is still asked in interviews and widely deployed.

5.8 VACUUM and Retention

★★★

IN ONE LINE

Deletes obsolete Delta files after the retention window, balancing storage cost against time travel and reader safety.

Delta never edits files in place: an `UPDATE`, `DELETE`, `MERGE` or `OPTIMIZE` writes new files and only marks the old ones as removed in the log. They remain on storage so **time travel** and concurrent readers keep working. **VACUUM** permanently deletes files that are no longer referenced **and** older than the retention threshold (default **7 days / 168 hours**).

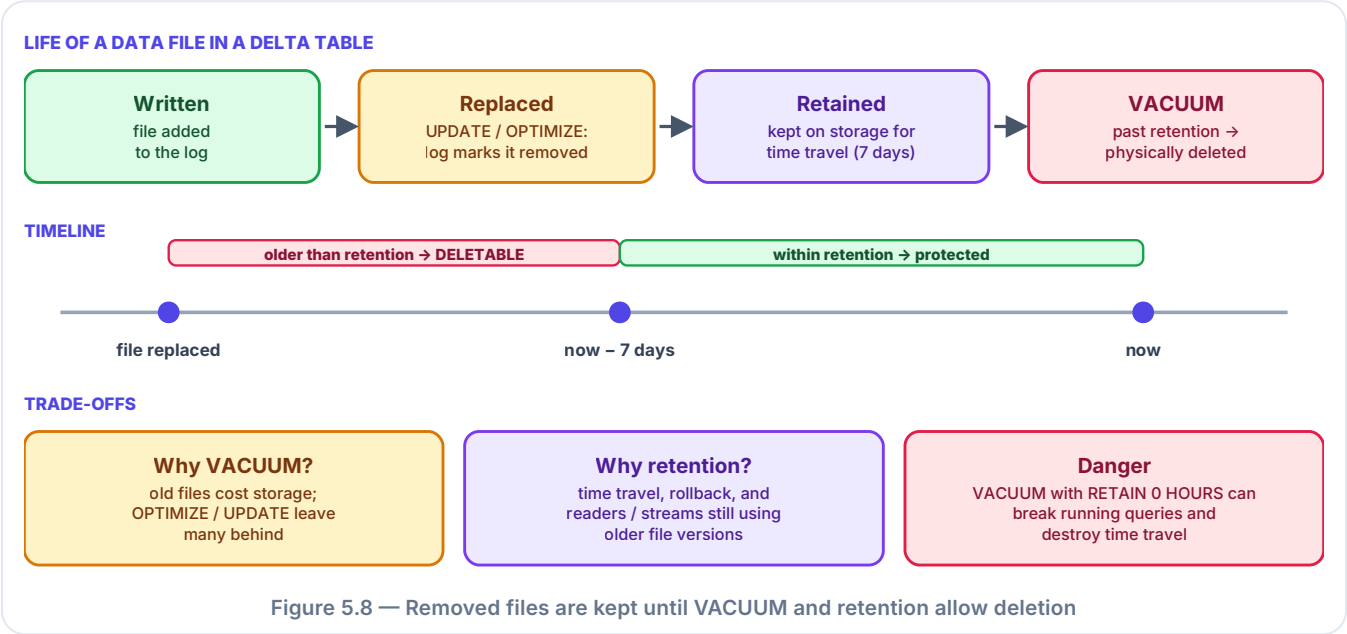


Figure 5.8 — Removed files are kept until VACUUM and retention allow deletion

VACUUM AND RETENTION

```
-- preview what would be deleted
VACUUM sales.orders DRY RUN;

-- delete eligible files (default 7 days)
VACUUM sales.orders;

-- explicit retention
VACUUM sales.orders RETAIN 336 HOURS;      -- 14 days

-- table properties that control retention
ALTER TABLE sales.orders SET TBLPROPERTIES (
  'delta.deletedFileRetentionDuration' = 'interval 14 days',
  'delta.logRetentionDuration'         = 'interval 60 days');
```

Consideration	Guidance
Retention shorter than longest-running query / stream	**Never**: readers can fail with FileNotFound.
Need to time travel / restore N days	Retention must be ≥ N days.
Regulatory deletion (GDPR)	DELETE then VACUUM after retention to remove data physically.
Schedule	Regularly (e.g. weekly) after OPTIMIZE; or let predictive optimization handle it.

COMMON TRAP

`VACUUM … RETAIN 0 HOURS` needs a safety check disabled and can corrupt in-flight reads and remove the ability to time travel. Do not do this on production tables.